

Cloud deployment

COMPLETE LEARNING GUIDE

What You'll Learn

Getting a full-stack application running on a laptop is a very different problem from running it reliably, securely, and scalably for real users in the cloud. This guide surveys the major deployment models (raw virtual machines, Platform-as-a-Service, serverless, and container orchestration) and the trade-offs between them, how to think about choosing a hosting platform for a typical full-stack app, the DNS/SSL layer that sits between a domain name and a running server, horizontal versus vertical scaling, keeping local and cloud environments consistent enough to trust testing, the monitoring/logging/alerting that turns 'find out from an angry user' into 'find out automatically,' and deploying changes without downtime.

1. Deployment Models: VM, PaaS, Serverless, Containers

A raw virtual machine (an EC2 instance, a DigitalOcean Droplet) gives full control over the operating system and runtime but means you're responsible for OS patching, process management, and scaling yourself. A Platform-as-a-Service (Render, Railway, Heroku-style platforms) takes a git push or a container and handles provisioning, scaling, and infrastructure for you, at the cost of less low-level control. Serverless (AWS Lambda, Vercel/Netlify functions) runs your code only in response to individual requests/events, scaling to zero when idle and billing per invocation, well-suited to spiky or infrequent workloads but with cold-start latency and execution-time limits to design around. Container orchestration (Kubernetes) manages many containers across a cluster of machines with fine-grained control over scaling, networking, and rollout strategy, at the cost of meaningfully higher operational complexity than the other options.

2. Choosing a Hosting Platform for a Full-Stack App

For most learners and small-to-mid production apps, the practical decision tree is: a PaaS (Render, Railway, Fly.io) for the backend API and a managed database service, plus a frontend-specialized platform (Vercel, Netlify) for a React/Next.js frontend, gets a full-stack app into production quickly with minimal infrastructure management — and this is genuinely the right choice for a large share of real applications, not just a beginner's shortcut. Kubernetes-based container orchestration earns its added complexity once an organization has enough services, enough scale, or specific compliance/control requirements that a managed PaaS genuinely can't satisfy — reaching for it by default, before that need is concrete, adds operational burden without a corresponding benefit.

3. DNS, Domains, and SSL/TLS

DNS translates a human-readable domain name into the IP address of the server actually handling requests, via records configured with your domain registrar or DNS provider — an A record points a domain directly to an IP, and a CNAME record points it to another domain name (common when pointing a custom domain at a hosting platform's own infrastructure). SSL/TLS certificates encrypt traffic between the browser and server and are what make a domain load over https instead of http; nearly every modern hosting platform (Vercel, Render, Cloudflare) provisions and renews these automatically via Let's Encrypt, removing what used to be a genuinely painful manual process.

4. Horizontal vs. Vertical Scaling

Vertical scaling means giving a single server more resources (more CPU, more RAM) — simple, but with a hard ceiling (there's a biggest machine you can rent) and no redundancy (if that one machine goes down, everything goes down). Horizontal scaling means running multiple smaller server instances behind a load balancer that distributes traffic across them — this is what enables both near-unlimited scaling and redundancy (one instance failing doesn't take down the whole service), but, as covered in server architecture, it requires the application to be stateless, since any instance might handle any given request.

5. Environment Parity Between Local and Cloud

The further local development diverges from the production environment (a different database engine locally versus in production, different Node.js versions, different OS-level behavior), the more 'works on my machine, breaks in production' surprises show up, precisely at the worst time. Using containers locally (matching the same Docker image or base image used in production) and matching dependency versions exactly via a lockfile are the two most effective, practical ways to keep environment parity high without literally running production-scale infrastructure on a laptop.

6. Monitoring, Logging, and Alerting in Production

Once an application is live, the question isn't whether something will eventually go wrong, but whether you find out from your own monitoring or from an angry user's support ticket. Monitoring tools (Datadog, Sentry, or a cloud platform's built-in metrics) track error rates, response times, and resource usage over time; centralized logging aggregates logs from every instance into one searchable place; and alerting rules notify the team automatically when a metric crosses a concerning threshold (error rate spikes, response time degrades, a health check starts failing) — the entire point being to be notified before most users are affected, not after.

7. Zero-Downtime Deploys and Rollbacks

A naive deploy that stops the old version and then starts the new one creates a visible gap where the service is completely down. A zero-downtime deploy starts new instances running the new version, waits until they pass a health check, then only shifts traffic to them and terminates the old instances afterward — most managed hosting platforms and container orchestrators support this pattern out of the box, requiring only that your application expose a health-check endpoint and handle graceful shutdown (finishing in-flight requests instead of dropping them when told to terminate). Having a fast, tested rollback path (redeploying the previous known-good version) is just as important as the deploy mechanism itself, since some issues only surface under real production traffic.

Common Pitfalls

- Having no monitoring or alerting at all, so outages and errors are first discovered through user complaints instead of automated detection.
- Deploying with hardcoded localhost URLs or development-only configuration that silently breaks once the app is actually running in the cloud.
- Forgetting to configure HTTPS/SSL for a production domain, or letting a certificate expire due to relying on manual renewal.
- Scaling application servers horizontally without checking the database's connection limit, causing 'too many connections' errors under load.
- Treating a cloud deployment as 'set and forget,' missing both cost creep from unused/oversized resources and slow performance regressions over time.
- Running a single server instance with no redundancy, creating a single point of failure for the entire application.
- Skipping a staging environment entirely and testing changes for the first time directly in production.

Key Takeaways

- PaaS platforms get most full-stack apps into production quickly; Kubernetes-style orchestration is justified only once scale or control requirements genuinely demand it.
- Modern hosting platforms automate SSL/TLS certificate provisioning and renewal via Let's Encrypt — manual certificate management is largely a solved problem now.
- Horizontal scaling requires a stateless application; vertical scaling has a hard resource ceiling and no built-in redundancy.
- High environment parity (matching containers, matching lockfile versions) between local and production prevents an entire class of late-stage surprises.
- Monitoring and alerting exist to catch problems before users do — treat their absence as a production gap, not an optional extra.

- A zero-downtime deploy needs a health check and graceful shutdown handling; a fast rollback path matters just as much as the deploy mechanism.

Learn More

MDN — What is a Domain Name / DNS

https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_domain_name

AWS — Types of Cloud Computing (overview of deployment models)

<https://aws.amazon.com/types-of-cloud-computing/>

Google SRE Book — Monitoring Distributed Systems

<https://sre.google/sre-book/monitoring-distributed-systems/>